



Ashling* RiscFree* Integrated Development Environment (IDE) for Intel® FPGAs User Guide

Updated for Intel® Quartus® Prime Design Suite: **23.2**



Online Version

Send Feedback

730783

2023.07.20

Contents

1. About the RiscFree* IDE.....	4
1.1. Supported Devices.....	4
1.2. Intel Quartus Prime Software Support.....	5
2. Installation and Setup.....	6
2.1. Bundled Installation with the Intel Quartus Prime Software.....	6
2.2. Standalone Installation.....	7
2.2.1. Standalone Installation with Supported Intel Quartus Prime Software.....	8
2.2.2. Standalone Installation with the Intel Quartus Prime Programmer and Tools.....	8
3. Getting Started with RiscFree IDE.....	10
3.1. Graphical User Interface.....	10
3.2. Command Line Interface.....	10
4. Debug Setup for Nios V Processor System.....	11
4.1. Importing Nios V Processor Project.....	11
4.2. Building Nios V Processor Project.....	13
4.3. Setting Debug Configurations and Downloading Nios V Processor Project Using RiscFree IDE.....	13
5. Debug Setup for Arm Hard Processor System.....	17
5.1. Pre-requisite for Debugging Arm HPS Project.....	17
5.2. Importing Arm HPS Project.....	17
5.3. Setting Debug Configurations and Downloading Arm HPS Project Using RiscFree IDE....	18
6. Debugging with RiscFree IDE.....	20
6.1. Debug Features in RiscFree IDE.....	20
6.1.1. Viewing Global Variables.....	20
6.1.2. Launching Memory Browser.....	20
6.1.3. Debugger Console.....	21
6.1.4. Specifying the SVD File.....	21
6.1.5. Breakpoint Settings.....	21
6.1.6. JTAG UART Output Terminal.....	22
6.2. Processor System Debug.....	22
6.2.1. GUI for Debugging.....	23
6.3. Heterogeneous Multicore Debug.....	23
6.3.1. Importing Nios V Processor and Arm HPS Projects.....	24
6.3.2. Setting Core Configuration.....	24
6.4. Debugging μ C/OS-II Application.....	30
6.5. Debugging FreeRTOS Application.....	30
6.6. Debugging Zephyr Application.....	31
6.7. Arm HPS On-Chip Trace.....	33
6.7.1. Trace Bar.....	37
6.7.2. Trace View.....	38
7. Ashling RiscFree Integrated Development Environment (IDE) for Intel FPGAs User Guide Archives.....	40
8. Document Revision History for the Ashling RiscFree Integrated Development Environment (IDE) for Intel FPGAs User Guide.....	41

A. Appendix.....	43
A.1. Optional Toolchain Configuration.....	43
A.1.1. Toolchain for Application and BSP.....	43
A.1.2. Toolchain for Arm Processor.....	43
A.1.3. Modifying Toolchain Configuration.....	44
A.2. Performing External Tools Configuration.....	45
A.3. Help Content.....	46
A.4. Open-Source Components in RiscFree IDE.....	46

1. About the RiscFree* IDE

RiscFree* is Ashling's Eclipse* C/C++ Development Toolkit (CDT) based integrated development environment (IDE) for Intel® FPGAs Arm*-based HPS and RISC-V based Nios® V processors.

The RiscFree IDE provides a complete, seamless environment for C and C++ software development and has the following features:

- Eclipse CDT based IDE with full source and project creation, editing, build, and debug support using the RISC-V GNU compiler collection (GCC) toolchain.
- Project Manager and Build Manager including Make and CMake support with rapid import, build, and debug of application frameworks created using the Intel Quartus® Prime software.
- RISC-V GNU GCC toolchain with support for newlib or picolibc run-time libraries using the Nios V Hardware Abstraction Layer (HAL) API for hardware access.
- Integrated support for Intel FPGA Download Cable II JTAG debug probe.
- ROM or RAM based debugging support, for example, hardware breakpoints for flash-based support.
- High-level Register Viewer based on industry standard System View Description (SVD) files.
- Integrated serial terminal.

1.1. Supported Devices

The following devices support Nios V core debugging:

- Intel Agilex® 7
- Intel Stratix® 10
- Intel Stratix V
- Intel Stratix IV
- Intel Arria® 10
- Intel Arria II GX
- Intel Arria II GZ
- Intel Arria V
- Intel Arria V GX
- Intel Cyclone® 10 GX
- Intel Cyclone IV E
- Intel Cyclone IV GX

- Intel Cyclone V
- Intel Cyclone 10 LP
- Intel MAX® 10

The following devices with SoC feature support Arm HPS debugging:

- Intel Agilex 7
- Intel Stratix 10
- Intel Arria 10
- Cyclone V

1.2. Intel Quartus Prime Software Support

You can use Ashling* RiscFree IDE for Intel FPGAs with Intel Quartus Prime Standard Edition or Intel Quartus Prime Pro Edition software.

Table 1. Nios V Processor Software Support

Nios V Processor	Intel Quartus Prime Standard Edition	Intel Quartus Prime Pro Edition
Intel Quartus Prime version	22.1	23.2
Ashling RiscFree for Intel FPGAs version	22.3.2	23.2.2

Related Information

- [Nios V Embedded Processor Design Handbook](#)
Describes how to most effectively use the tools, recommends design styles, and practices for developing, debugging, and optimizing embedded systems using the Nios V processor and Intel-provided tools (Intel Quartus Prime Pro Edition User Guide).
- [Nios V Processor Reference Manual](#)
Provides information about the Nios V processor performance benchmarks, processor architecture, the programming model, and the core implementation (Intel Quartus Prime Pro Edition User Guide).
- [Nios V Processor Software Developer Handbook](#)
Describes the Nios V processor software development environment, the tools that are available, and the process to build software to run on Nios V processor (Intel Quartus Prime Pro Edition User Guide).
- [AN 980: Nios® V Processor Intel® Quartus® Prime Software Support](#)



2. Installation and Setup

RiscFree IDE supports Nios V processor and Arm core debugging in multiple host operating system (OS) platforms. Refer to the *Operating System Support* page for the OS supported.

Note: If you install both the Intel Quartus Prime software and the Intel Quartus Prime Programmer and Tools in the same location as RiscFree IDE, RiscFree IDE refers to the required binaries from the Intel Quartus Prime software installation directory by default.

RiscFree IDE has two types of installation options:

- Bundled installation option when installing the Intel Quartus Prime software
- Standalone installation which requires you to install one of the following to use the debugger:
 - The Intel Quartus Prime Programmer and Tools
 - The Intel Quartus Prime software

Related Information

[Operating System Support](#)

2.1. Bundled Installation with the Intel Quartus Prime Software

The RiscFree IDE is available in the Intel Quartus Prime software installation package as an optional installation component. For a bundled installation, select the **RiscFree IDE for Intel FPGAs** component during the Intel Quartus Prime Pro Edition software installation.

Note: The RiscFree for Intel FPGAs is not bundled with the 22.1std release of Intel Quartus Prime Standard Edition. Refer to the following sections for the RiscFree IDE for Intel FPGAs standalone installation.

Figure 1. Selecting Components During Bundled Installation

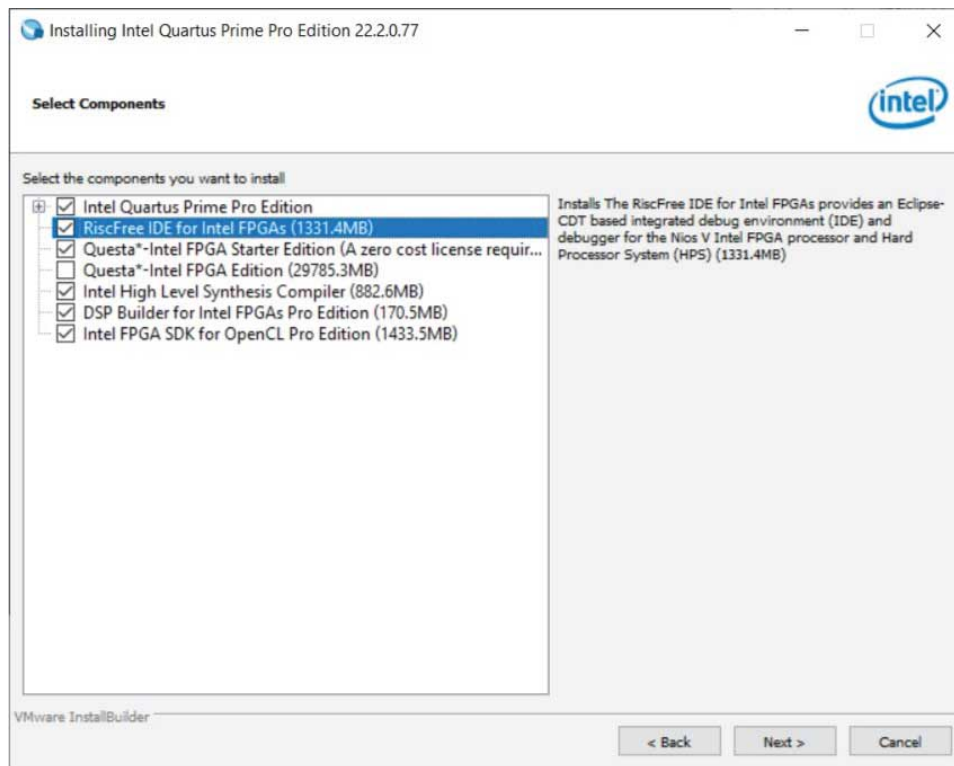


Table 2. Installation Directory for Bundled Installation with the Intel Quartus Prime Software

Operating System	Linux*	Windows*
RiscFree IDE Installation Directory	<Intel Quartus Prime installation directory>/riscfree/	<Intel Quartus Prime installation directory>/riscfree/
Intel Quartus Prime Binary Files Directory	<Intel Quartus Prime installation directory>/quartus/linux64/...	<Intel Quartus Prime installation directory>/quartus/bin64/...

Related Information

[FPGA Software Download Center](#)

Provides the latest version of the Intel Quartus Prime Design Software.

2.2. Standalone Installation

You can download the Ashling RiscFree IDE for Intel FPGAs from the *FPGA Software Download Center*.

1. Select the latest version of Intel Quartus Prime design software.
2. Scroll down to *Downloads*, and click the *Additional Software* tab.
3. Scroll down to *Stand-Alone Software*.
4. Find Ashling RiscFree IDE for Intel FPGAs and click *Download*.

Related Information

[FPGA Software Download Center](#)

Provides the latest version of the Intel Quartus Prime Design Software.

2.2.1. Standalone Installation with Supported Intel Quartus Prime Software

You can install the standalone RiscFree IDE for your system that has a pre-installed Intel Quartus Prime software.

The standalone RiscFree IDE debug utility requires the Intel Quartus Prime binaries from the installed Intel Quartus Prime software directory: `../<Intel Quartus Prime installation directory>`. Install the RiscFree IDE in the same path as the Intel Quartus Prime software. If you select a different path, you receive the following message:

Figure 2. Installation Warning Message

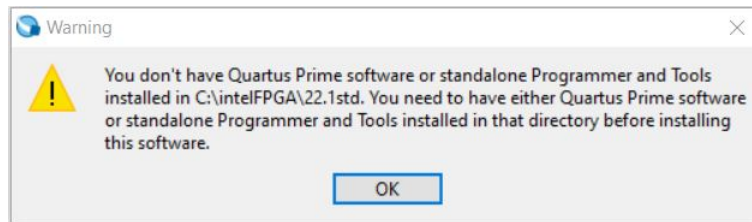


Table 3. Installation Directory for Standalone Installation with the Intel Quartus Prime Software

Operating System	Linux	Windows
RiscFree IDE Installation Directory	<code><Intel Quartus Prime installation directory>/riscfree/</code>	<code><Intel Quartus Prime installation directory>/riscfree/</code>
Intel Quartus Prime Software Directory	<code><Intel Quartus Prime installation directory></code>	<code><Intel Quartus Prime installation directory></code>

2.2.2. Standalone Installation with the Intel Quartus Prime Programmer and Tools

You can install the standalone RiscFree IDE with the Intel Quartus Prime Programmer and Tools.

The standalone RiscFree IDE debug utility requires the Intel Quartus Prime Programmer and Tools binaries from the installed Intel Quartus Prime Programmer and Tools directory. Install the RiscFree IDE in the same path as the Intel Quartus Prime Programmer and Tools: `../<Quartus_programmer_installation_directory>`. If you select a different path, you receive an error message. Refer to [Figure Installation Warning Message](#) for the error message.

Table 4. Installation Directory for Standalone Installation with the Intel Quartus Prime Programmer and Tools

Operating System	Linux	Windows
RiscFree IDE Installation Directory	<Quartus_programmer_installation_directory>/riscfree/	<Quartus_programmer_installation_directory>/riscfree/
Intel Quartus Prime Programmer Directory	<Quartus_programmer_installation_directory>/qprogrammer/quartus/linux64/...	<Quartus_programmer_installation_directory>/qprogrammer/quartus/bin64/...

Note: Nios V processor tools are not available for standalone installation with the Intel Quartus Prime Programmer and Tools.

3. Getting Started with RiscFree IDE

3.1. Graphical User Interface

You can run RiscFree IDE launcher from the installed directory.

Note: On Windows, a **RiscFree IDE for Intel FPGAs** Start Menu shortcut is created. You can launch RiscFree IDE using this shortcut from the Windows Start Menu.

Table 5. RiscFree IDE Launcher Location

Operating System	RiscFree IDE Launcher Location
Linux	<Intel Quartus Prime installation directory>/ riscfree/RiscFree/RiscFree
Windows	<Intel Quartus Prime installation directory>/ riscfree/RiscFree/RiscFree.exe

If this is your first time running the RiscFree IDE, browse and select your Workspace default directory for projects and click **Launch**.

3.2. Command Line Interface

RiscFree IDE supports command-line invocation. You can launch RiscFree IDE using `niosv-shell`.

Table 6. Syntax to Invoke RiscFree IDE Using Command-Line

Operating System	Syntax
Linux	<code>./RiscFree -data <Workspace Path></code>
Windows	<code>RiscFree.exe -data <Workspace Path></code>

Note: For standalone installation, to avoid the `niosv-stack-report` warning when building the "Hello World" application, launch RiscFree IDE using `niosv-shell`. For standalone installation, you may see the `niosv-stack-report` warning if you launch RiscFree IDE using GUI.

Related Information

KDB link: [Why does niosv-stack-report return error during Nios V build flow in RiscFree IDE for Intel FPGAs?](#)

4. Debug Setup for Nios V Processor System

You can develop software for the Nios V processor effectively using Intel FPGA development kit. The development kit includes documentation, a ready-made evaluation board, a getting-started reference design, and all the development tools necessary to write Nios V processor programs.

Modifying existing code is a common, easy way to learn to start writing software in a new environment. The Nios V Processor Intel FPGA IP provides example software designs that you can examine, modify, and use in your own programs. The examples are documented in the *Nios V Embedded Processor Design Handbook* and are ready to compile.

This section guides you through the fundamental operations in RiscFree IDE for Nios V processors, providing steps for the following applications:

- Importing an application project for the Nios V processor, with the board support package (BSP) project required to interface with your hardware.
- Building an application and BSP projects in RiscFree IDE.
- Running the software on an Intel FPGA development board.

The following operating systems support Nios V processor applications:

- Intel HAL (Bare-Metal)
- MicroC/OS-II (μC/OS-II)
- Zephyr®
- FreeRTOS™

Related Information

- [Nios V/m Processor Example Design section, Nios V Embedded Processor Design Handbook](#)
Provides the Nios V processor example design.
- [Generating the Application Project File section, Nios V Embedded Processor Design Handbook](#)
Provides the steps to select your preferred OS type for the application.

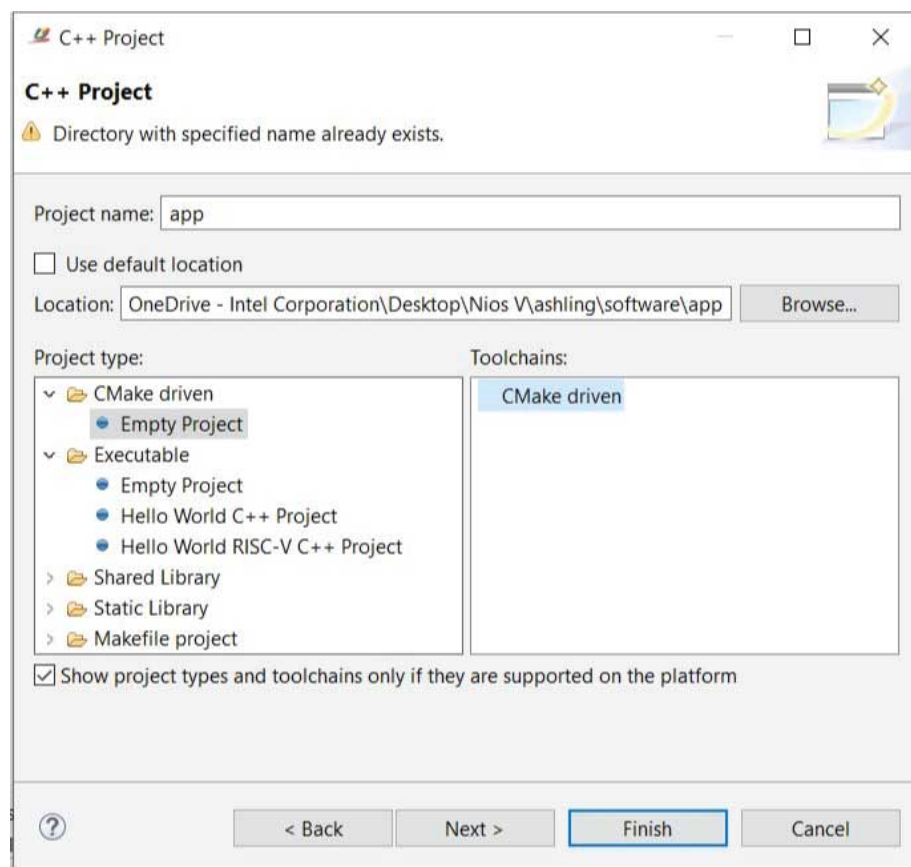
4.1. Importing Nios V Processor Project

Note: To generate the Nios V processor BSP or application project, refer to the [Nios V Embedded Processor Design Handbook](#).

To import Nios V processor BSP and application project to RiscFree IDE, follow these steps:

1. To open the **New Project** wizard, click **File > New > Project....**
2. In the **New Project** wizard, click **C/C++ > C++ Project** to select project.
3. In **C++ Project** wizard, turn off **Use default location**. Browse and select the location of the Nios V processor BSP or application project. Enter the project name (use the same name as the project folder).
4. Under **Project type**, select **CMake driven > Empty Project**. Under **Toolchains**, select **CMake driven**. Click **Finish**. The CMake driven application or BSP project is added to the Project Explorer.

Figure 3. Importing Nios V Processor Sample Project



Related Information

[Nios V Embedded Processor Design Handbook](#)

Provides the Nios V processor quick start guide.

4.2. Building Nios V Processor Project

You can browse the project files for Nios V processor project in RiscFree IDE Project Explorer tab. To build the project, follow these steps:

1. Right-click the project folder and click **Build Project**.
2. When the build is complete, you can find the console prints as shown in the following diagram.

Figure 4. Console Prints



```

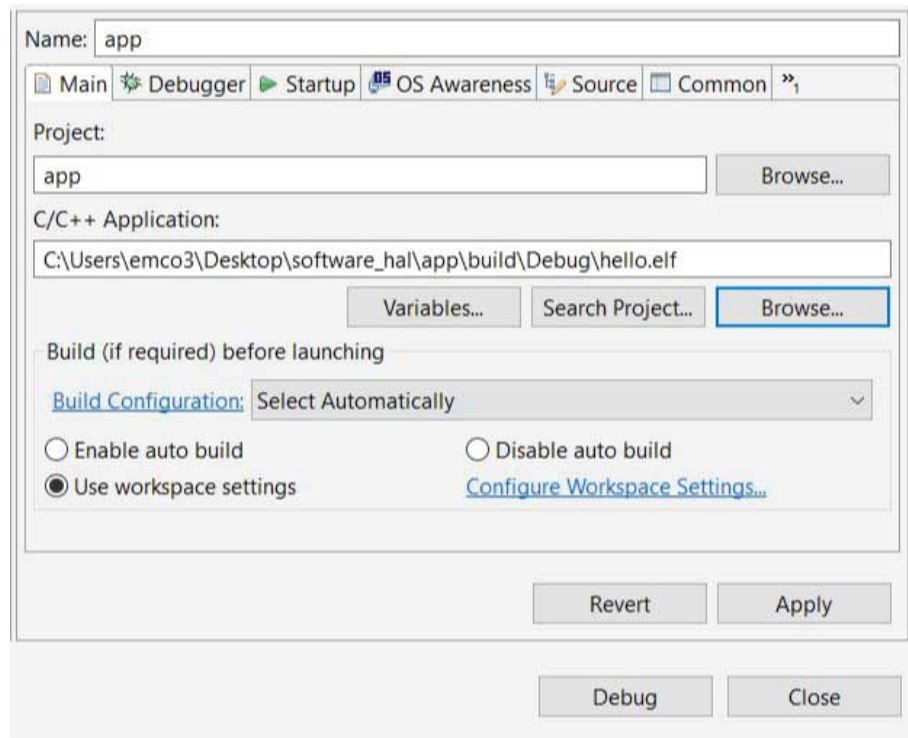
CMake Console [app]
12:01:26 Buildscript generation: app::Debug in C:\Users\emco3\Desktop\software\app\build\Debug
cmake -DCMAKE_BUILD_TYPE=Debug -DCMAKE_EXPORT_COMPILE_COMMANDS=ON -G "Unix Makefiles" "C:\Users\emco3\Desktop\software\app"
-- The ASM compiler identification is GNU
-- Found assembler: C:/intelFPGA_pro/22.2/riscfree/toolchain/riscv32-unknown-elf/bin/riscv32-unknown-elf-gcc.exe
-- The C compiler identification is GNU 11.2.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: C:/intelFPGA_pro/22.2/riscfree/toolchain/riscv32-unknown-elf/bin/riscv32-unknown-elf-gcc.exe - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- The CXX compiler identification is GNU 11.2.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: C:/intelFPGA_pro/22.2/riscfree/toolchain/riscv32-unknown-elf/bin/riscv32-unknown-elf-g++.exe - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: C:/Users/emco3/Desktop/software/app/build/Debug
12:01:31 Buildscript generation finished (took 4899 ms)
  
```

4.3. Setting Debug Configurations and Downloading Nios V Processor Project Using RiscFree IDE

You can download and debug a Nios V processor software project on the targeted Intel FPGA using the RiscFree IDE. To debug the project, follow these steps:

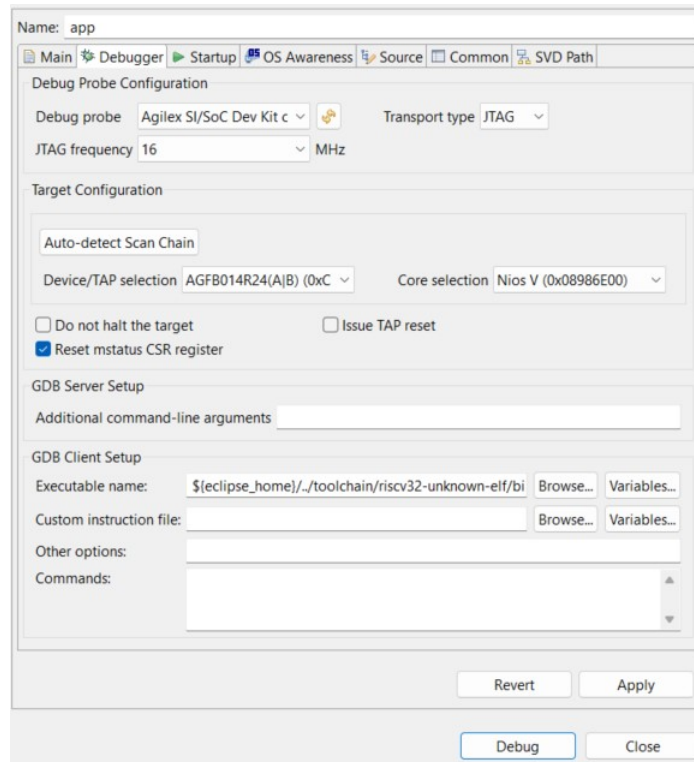
1. Right-click the project folder (application or BSP) in the project explorer and select **Debug As ► Debug Configurations**.
2. Select **Ashling RISC-V (auto-detect) Hardware Debugging ► <PROJECT_NAME>**. Ensure the **Project** and **C/C++ Application** match with your project name and your project .elf file respectively.
3. Under the **Main** tab, for **C/C++ Application**, browse to select the application build .elf file. For example: hello.elf.

Figure 5. Debug Configurations for Nios V Processor –Main Tab



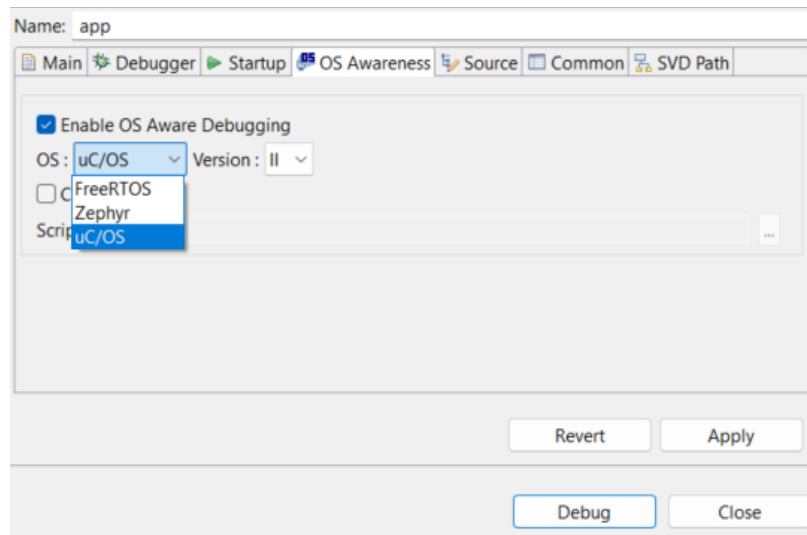
4. Under the **Debugger** tab, set these settings:
 - Debug probe: **1 (USB-Blaster II)**
 - Transport type: **JTAG**
 - JTAG frequency: **16 MHz**

Figure 6. Debug Configurations for Nios V Processor —Debugger Tab



5. Click **Auto-detect Scan Chain** to automatically detect JTAG scan chain information of the target device. Select the options from **Device/Tap selection** and **Core selection**.
6. Based on the OS you use, configure the OS Awareness settings as follows:
 - Intel HAL: No OS Awareness configuration is required.
 - Other OS: Under the **OS Awareness** tab, turn on **Enable OS Aware Debugging**, and select the OS version applicable to you as listed below:
 - **µC/OS** and Version: **II**
 - **FreeRTOS** and Version: **10.4.1**
 - **Zephyr** and Version: **3.1.0**

Figure 7. Enabling OS Aware Debugging in RISC-V Hardware Debugging



7. Click **Debug**. RiscFree IDE downloads the program to the target and you can find the console prints as shown in the following diagram.

Figure 8. Console Prints after Debug Connection is Successful

```

app [Ashling RISC-V (auto-detect) Hardware Debugging]
Ashling GDB Server for RISC-V (ash-riscv-gdb-server).
v22.2.0, 06-May-2022, (c)Ashling Microsystems Ltd 2022.

Initializing connection ...
Checking for an active debug connection using the selected debug probe (SN: 1):
Connected to target device "" using USB-Blaster-2 (1) via JTAG at 16MHz.
Info : Active Harts Detected : 1
Info : [0] System architecture : RV32
Info : [0] Number of hardware breakpoints available : 1
Info : [0] Number of program buffers: 8
Info : [0] Number of data registers: 2
Info : [0] Memory Access -> Program buffer
Info : [0] Memory Access -> Abstract access memory
Info : [0] CSR & FP Register Access -> Abstract commands

Waiting for debugger connection on port 58465 for core 1.
Press 'Q' to Quit.
Got a debugger connection from 127.0.0.1 on port 58465.

```

8. Refer to the *Debugging with RiscFree IDE* section for further debugging.

Note: You can issue a debug reset using `niosv-download -r` command. This command only resets the Nios V processor if the debug reset interface is connected to the Nios V processor IP's reset input in your Platform Designer.

Note: `niosv-download` (under `<Intel Quartus Prime installation directory>/niosv/bin` directory) is only available for the Intel Quartus Prime software. This tool is not available for standalone RiscFree IDE installation with the Intel Quartus Prime Programmer and Tools.

Related Information

[Debugging with RiscFree IDE](#) on page 20

5. Debug Setup for Arm Hard Processor System

This section provides guidance on debugging Arm HPS core using the RiscFree IDE debugging tool and accessing the hardware modules (for example, peripherals) that are instantiated in the RiscFree IDE FPGA design. This tutorial is based on the Intel FPGA example design from Ashling.

Note: RiscFree IDE supports Arm HPS standalone debug.

5.1. Pre-requisite for Debugging Arm HPS Project

This example uses the Intel Agilex 7 board. To prepare the platform you are planning to debug, follow these steps:

1. Prepare a golden system reference design (GSRD) HPS .sof file or use a prebuilt HPS .sof file.
2. Download the HPS .sof file to the Intel Agilex 7 board using the Intel Quartus Prime Programmer and Tools.

Related Information

- [Intel Agilex 7 SoC Golden System Reference Design \(GSRD\)](#)
Provides the prebuilt .sof files at the Release Contents section for download.
- [Cyclone V SoC Golden System Reference Design \(GSRD\)](#)
Provides the prebuilt .sof files at the Release Contents section for download.
- [Intel Stratix 10 SoC Golden System Reference Design \(GSRD\)](#)
Provides the prebuilt .sof files at the Release Contents section for download.
- [Intel Arria 10 SoC Golden System Reference Design \(GSRD\)](#)
Provides the prebuilt .sof files at the Release Contents section for download.

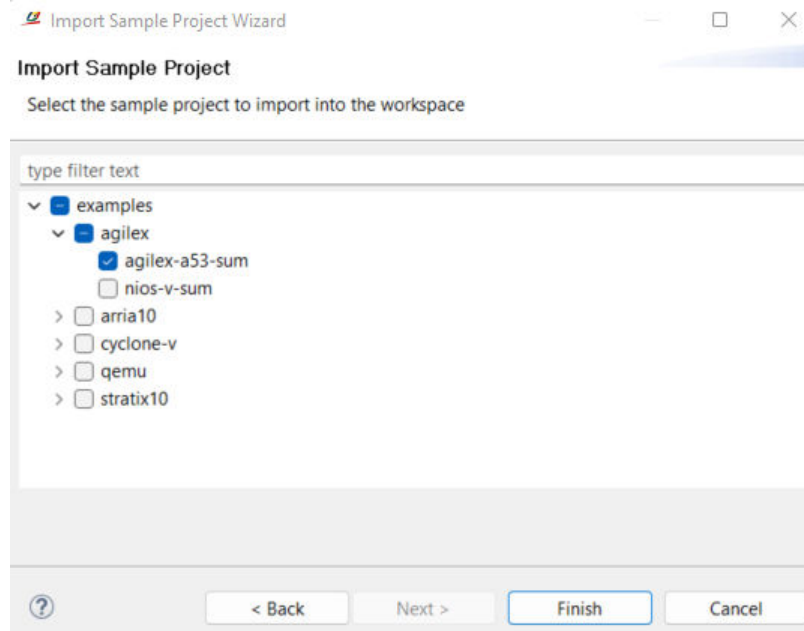
5.2. Importing Arm HPS Project

To import the Intel Agilex 7 Arm Cortex*-A53 sample project and a pre-built .elf file, follow these steps:

Note: Arm toolchain is not included by default. Follow the steps in the *Toolchain for Arm Processor* section to modify toolchain configuration to include Arm GNU GCC toolchain.

1. To open the **Import** wizard, click **File > Import**. In the **Import** wizard, select **Ashling Sample Projects > C/C++ Sample Projects** and click **Next**.
2. In the **Import Sample Project Wizard**, select **examples > agilex > cortex-a53-sum** and click **Finish**. The project is listed in the Project Explorer.

Figure 9. Importing Cortex-A53 Sample Project to RiscFree IDE



Related Information

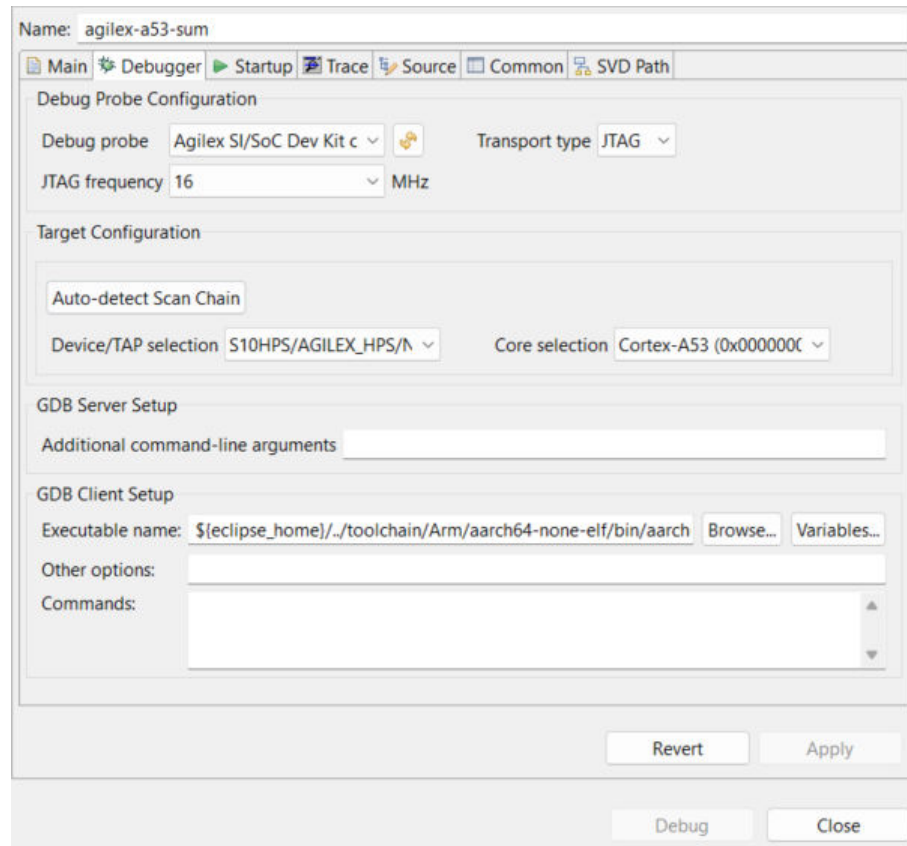
Toolchain for Arm Processor on page 43

5.3. Setting Debug Configurations and Downloading Arm HPS Project Using RiscFree IDE

To debug the project, follow these steps:

1. Right-click the project directory and select **Debug > Debug Configurations**.
2. Select **Ashling Arm Hardware Debugging > cortex-a53-sum**. Ensure the **Project** and **C/C++ Application** match with your project name and your project .elf file respectively.
3. Under the Debugger tab, set these settings:
 - Debug probe: **Agilex SI/SoC Dev Kit** — Select the connected device
 - Transport type: **JTAG**
 - JTAG/SWD frequency: **16 MHz**
 - Core selection: Can select any core between **0** to **3**

Figure 10. Debugger Settings for Intel Agilex 7 Arm Cortex-A53 Core



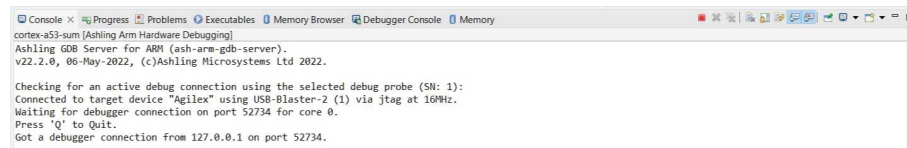
Note: The default settings for the current example design in the RiscFree IDE are configured based on Intel Agilex 7 device.

Note: If you are using a different Arm architecture, select the Arm GNU debugger (GDB) (select **GDB Client Setup ► Executable name**) manually.

- 64-bit ELFs: `$(eclipse_home)/../toolchain/Arm/aarch64-none-elf/bin/aarch64-none-elf-gdb.exe`
- 32-bit ELFs: `$(eclipse_home)/../toolchain/Arm/arm-none-eabi/bin/arm-none-eabi-gdb.exe`

4. Click **Debug**. RiscFree IDE downloads the program to the target and you can find the console prints as shown in the following diagram.

Figure 11. RiscFree IDE after Program is Downloaded and System is Ready for Debug



Related Information

Debugging with RiscFree IDE on page 20

6. Debugging with RiscFree IDE

6.1. Debug Features in RiscFree IDE

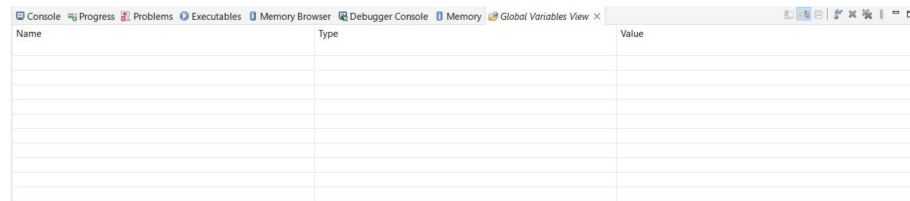
6.1.1. Viewing Global Variables

RiscFree IDE has dedicated global variables.

To view the global variables in the application (.elf), follow these steps:

1. Go to **Window ► Show View ► Other... ► Debug ► Global Variables View**.

Figure 12. Global Variables View



2. To list all available global variables in the application (.elf), in the **Global Variables View** tab, click the **Add Global Variables** icon. Select the required variables to view in the **Global Variables View**.

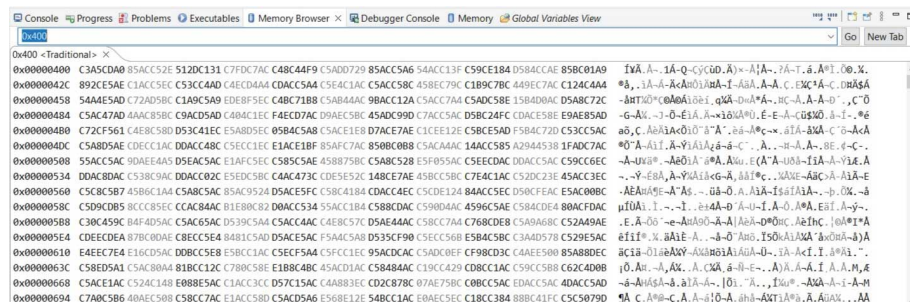
6.1.2. Launching Memory Browser

RiscFree IDE supports memory browser.

To launch the memory browser, follow these steps:

1. Go to **Window ► Show View ► Memory Browser**.
2. View the target memory in the memory browser.

Figure 13. Memory Browser showing Target Memory



6.1.3. Debugger Console

RiscFree IDE supports debugger console.

You can launch the debugger console from **Window ► Show View ► Debugger Console**.

You can enter debug commands and view output in the debug console. The GNU GDB syntax is fully supported in the console.

6.1.4. Specifying the SVD File

RiscFree IDE supports device-specific System View Description (SVD) JSON format files which provide a high-level, textual description of registers and the associated bits. Intel provides the SVD files as part of the Software Development Kit.

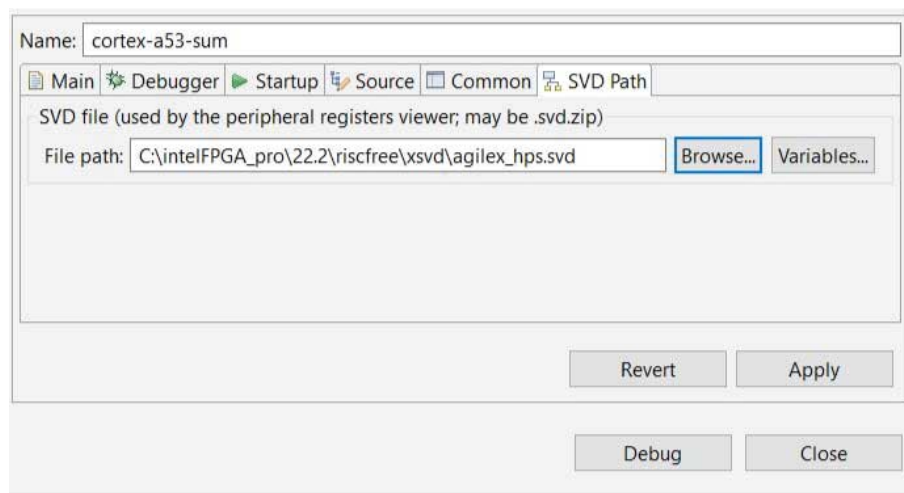
Table 7. SVD File Path

Operating System	SVD File Path
Linux	<Intel Quartus Prime installation directory>/riscfree/xsvd
Windows	<Intel Quartus Prime installation directory>/riscfree/xsvd

To specify the SVD file in Debug Configuration, follow these steps:

1. Open the **Debug Configuration** window.

Figure 14. Specifying the SVD File



2. Under the **SVD Path** tab, browse and select the SVD file.
3. Click **Debug** to start debugging.

6.1.5. Breakpoint Settings

RiscFree IDE supports SoC-wide breakpoint and core-specific software breakpoints in shared code.

SoC-wide Breakpoint

A single breakpoint can halt all cores in active debug launches. To enable SoC-wide breakpoint, go to **Run ► Enable SoC Wide Breakpoint**.

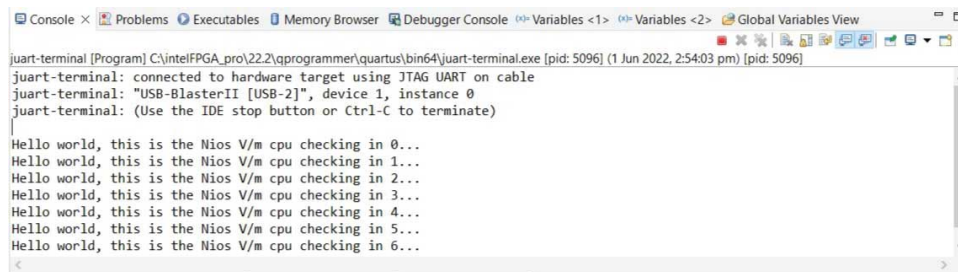
Core-specific Software Breakpoints

This breakpoint can only halt a specified core if debug is running for common code shared between multiple cores. To enable this feature, go to **Window ► Preferences ► C/C++ ► Debug ► GDB** and select **Set breakpoint for active debug context only**. This option is unchecked by default. If this is unchecked, any breakpoint set in shared code is applicable for all the cores sharing the code.

6.1.6. JTAG UART Output Terminal

You can view the output of JTAG UART (juart) terminal in the RiscFree IDE via the external tool configuration.

Figure 15. JTAG UART Terminal Console Print



```

juart-terminal [Program] C:\intelFPGA_pro\22.2\qprogrammer\quartus\bin64\juart-terminal.exe [pid: 5096] (1 Jun 2022, 2:54:03 pm) [pid: 5096]
juart-terminal: connected to hardware target using JTAG UART on cable
juart-terminal: "USB-BlasterII [USB-2]", device 1, instance 0
juart-terminal: (Use the IDE stop button or Ctrl-C to terminate)

Hello world, this is the Nios V/m cpu checking in 0...
Hello world, this is the Nios V/m cpu checking in 1...
Hello world, this is the Nios V/m cpu checking in 2...
Hello world, this is the Nios V/m cpu checking in 3...
Hello world, this is the Nios V/m cpu checking in 4...
Hello world, this is the Nios V/m cpu checking in 5...
Hello world, this is the Nios V/m cpu checking in 6...

```

Related Information

[Performing External Tools Configuration](#) on page 45

6.2. Processor System Debug

This section is a generic debugging section where the debugging steps are the same for Nios V processor and Arm HPS debugging.

Follow the guidelines in this section to start debugging the program using GUI or the GDB console after completing the steps from the following sections:

- [Setting Debug Configurations and Downloading Nios V Processor Project Using RiscFree IDE](#)
- [Setting Debug Configurations and Downloading Arm HPS Project Using RiscFree IDE](#)

Related Information

- [Setting Debug Configurations and Downloading Nios V Processor Project Using RiscFree IDE](#) on page 13
- [Setting Debug Configurations and Downloading Arm HPS Project Using RiscFree IDE](#) on page 18

6.2.1. GUI for Debugging

You can control the debugging process using the Debug Bar.

Figure 16. Execution Control in the Debug Bar



Table 8. Execution Control Using the Debug Bar

Debug Bar GUI	Function	Description
1	Resume	Continues the debugging process until the next breakpoint, or until the end of the debugging process.
2	Suspend	Halts the running code while debugging.
3	Terminate	Terminates the debug session. You need to relaunch the debug session after terminated.
4	Disconnect	Disconnects the debug tool connected to the RiscFree IDE.
5	Step into	Steps into the next method call at the currently executing line of code.
6	Step over	Steps over the next method call (without entering it) at the currently executing line of code. The method is still executed.
7	Step return (out)	Returns from a method which has been stepped into. The remainder of the code that was skipped by returning is still executed.
8	Instruction stepping mode	Switches to instruction stepping mode.
9	Drop to frame	Pops the current stack frame and puts control back out to the calling method, resetting any local variables.
10	Use step filters	Changes whether step filters should be used in the current Debug View.

6.3. Heterogeneous Multicore Debug

Heterogeneous multicore debugging is a method where you can debug two or more cores that differ in architecture or micro-architecture (Nios V processor core and Arm HPS core) simultaneously in the same integrated development environment.

By using heterogeneous multicore debugging, you can do the following:

- Simultaneously debug the entire integrated system in a single IDE.
- Real-time control of multiple processor cores that interact with each other.

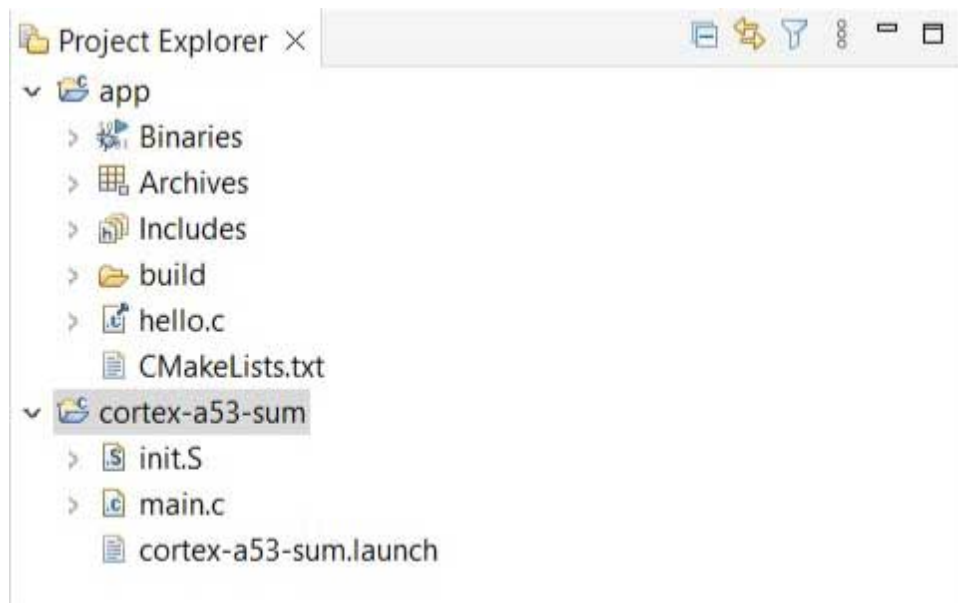
The example uses Nios V processor "Hello World" project and HPS design example provided by Ashling.

6.3.1. Importing Nios V Processor and Arm HPS Projects

To import the Nios V processor and Arm HPS projects to the RiscFree IDE workspace, follow these steps:

1. Follow the steps in the *Importing Nios V Project* and *Importing Arm HPS Project* sections.
2. After you import the projects, you can find the project directory in the Project Explorer.

Figure 17. Nios V Processor and Arm Cortex-A53 Ashling Samples Imported into RiscFree IDE Workspace



Related Information

- [Importing Nios V Processor Project](#) on page 11
- [Importing Arm HPS Project](#) on page 17

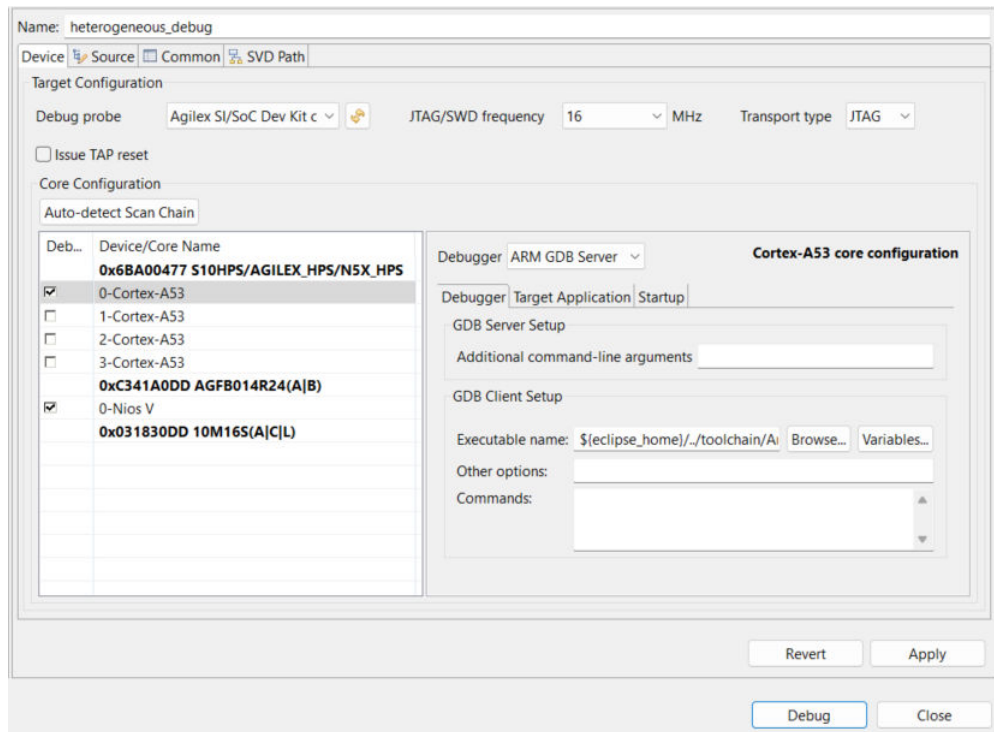
6.3.2. Setting Core Configuration

To set the core configuration, follow these steps:

1. Right click project directory (either HPS or Nios V application) and select **Debug > Debug Configurations**.
2. Select **Ashling Heterogenous Multicore Debugging > cortex-a53-sum.elf**.
3. Under the **Device** tab, set these settings:
 - Debug probe: **Agilex SI/SoC Dev Kit**
 - JTAG/SWD frequency: **16 MHz**
 - Transport type: **JTAG**
4. Click the **Auto-detect Scan Chain** to list out all the available cores.

- Under **Core Configuration**, select **Cortex-A53** and **Nios V/m**.

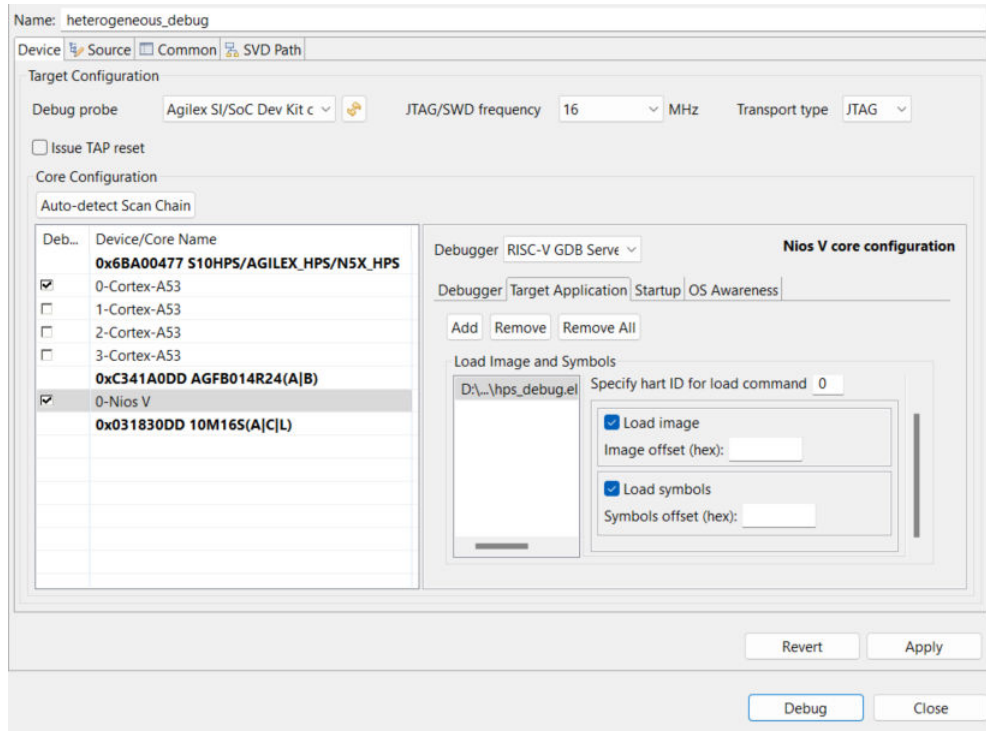
Figure 18. List of Available Cores for the Selected Target Device in RiscFree IDE



- Set the core specific configuration for each core under the **Debugger** tab.
- Specify the .elf file for each core under the **Target Application** tab. In this example, the target applications are added as nios-v-sum.elf for **Nios V/m** core and cortex-a53-sum.elf for **Cortex-A53** core.

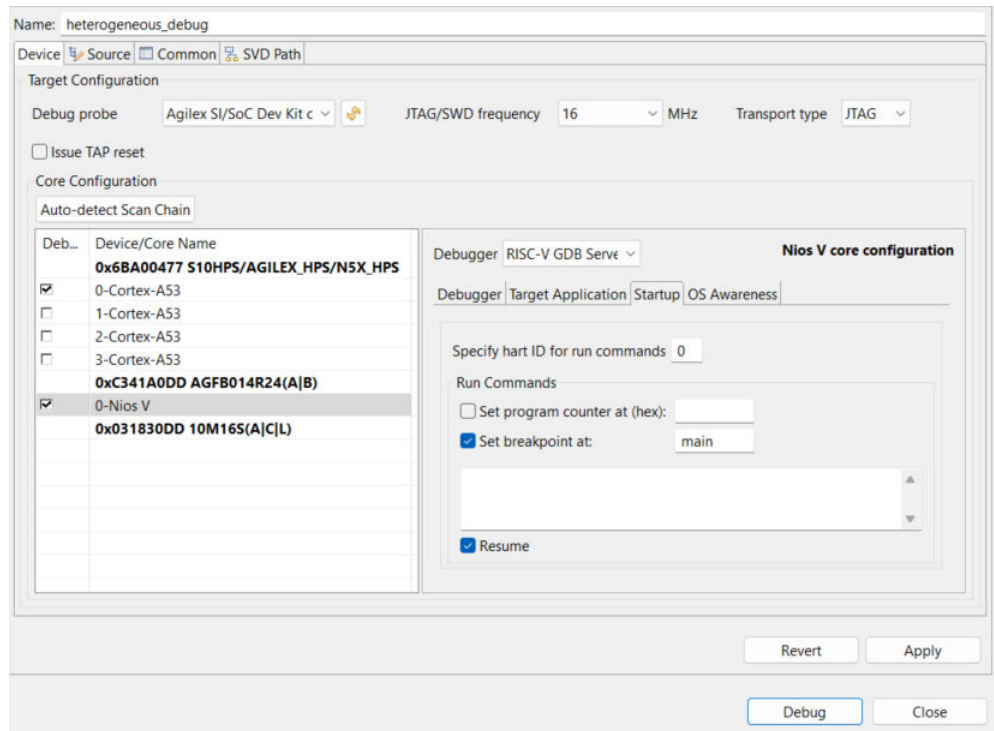
Note: You can specify multiple .elf file for a single core.

Figure 19. Target Application Tab Settings



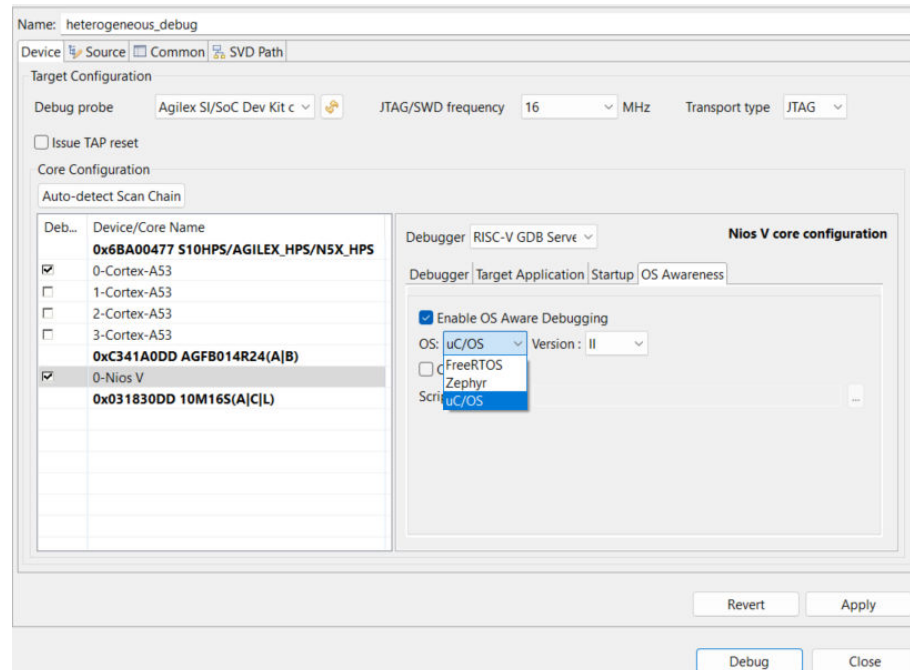
8. Set the breakpoint at a specific function under the **Startup** tab. This debug example uses the default settings.

Figure 20. Startup Tab Settings



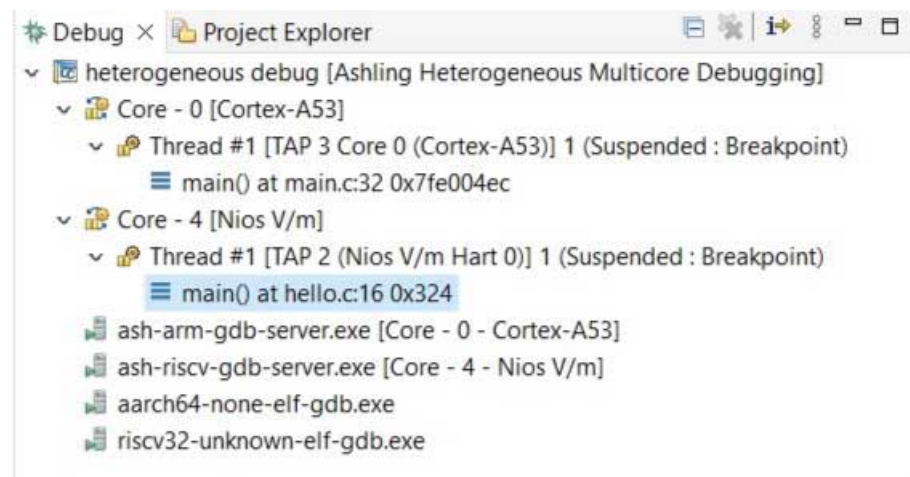
9. Based on the OS you use, configure the OS Awareness settings as follows:
 - Intel HAL: No OS Awareness configuration is required.
 - Other OS: Under the **OS Awareness** tab, turn on **Enable OS Aware Debugging**, and select the OS as listed below:
 - **µC/OS** and Version: **II**
 - **FreeRTOS** and Version: **10.4.1**
 - **Zephyr** and Version: **3.1.0**

Figure 21. Enabling OS Aware Debugging in Heterogeneous Debugging



10. Click **Debug** when all configuration is complete. All cores selected for debug are launched.

Figure 22. Heterogeneous Multicore Debug View in RiscFree IDE



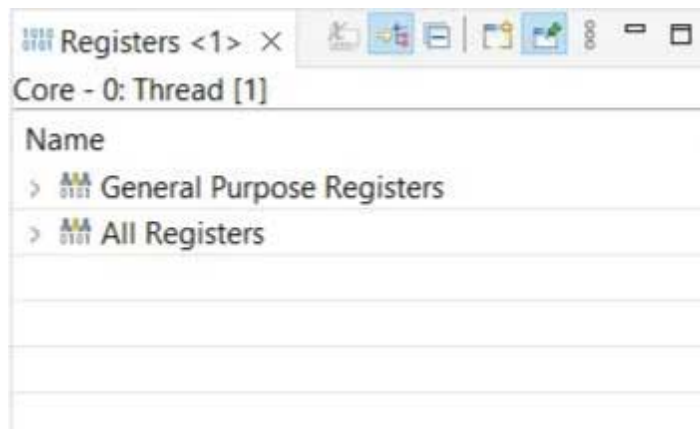
11. To debug two cores simultaneously, go to **Window > Show View > Registers**. Then, make a copy of the existing debug view using the **Open new View** icon.

Figure 23. Register Window



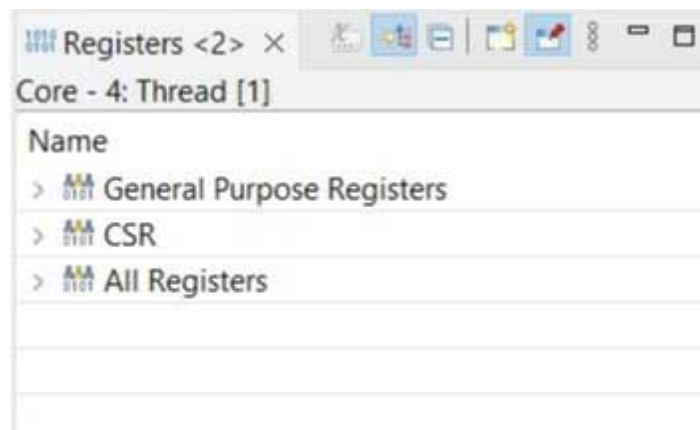
12. In the **Heterogenous Multicore Debug View** pane, select **Thread #1 [TAP 3 Core 0 (Cortex-A53)]**. Pin one copy of the debug view to Arm processor using the **Pin to Debug Context** icon.

Figure 24. Arm Processor (Core 0) Register View



13. In the **Heterogenous Multicore Debug View** pane, select **Thread #1 [TAP 2 (Nios V/m Hart 0)]**. Pin second copy of the debug view to Nios V processor using the **Pin to Debug Context** icon.

Figure 25. Nios V Processor (Core 4) Register View



6.4. Debugging μ C/OS-II Application

When you debug the μ C/OS-II application, you can view the number of tasks, task details, and queue list.

To debug μ C/OS-II application, follow these steps:

1. Launch the μ C/OS-II application.
 - To open the μ C/OS Task List, go to **Window > Show View > Other > μ C/OS > μ C/OS Task List**

Figure 26. μ C/OS Task List View

Task Name	Number	Priority	Start of Stack	Top of Stack	Status
uC/OS-II Idle	65535	20	0x2d19c	0x2d528	Ready to run
uC/OS-II Stat	65534	19	0x2c99c	0x2cd28	Ready to run

The μ C/OS Task List shows task details including task name, priority, status, and stack details.

- To open the μ C/OS Queue List, go to **Window > Show View > Other > μ C/OS > μ C/OS Queue List**.

Figure 27. μ C/OS Queue List View

Event Name	Event Type	Count	Related Task Count	Event Address
?	Semaphore	1	0	0x2c3ac
?	Semaphore	1	0	0x2c3c0
?	Semaphore	1	0	0x2c3d4
?	Semaphore	1	0	0x2c3e8
?	Semaphore	1	0	0x2c3fc

The μ C/OS Queue List shows details of all events and queues, including tasks related to each event.

6.5. Debugging FreeRTOS Application

When you debug the FreeRTOS application, you can view the number of tasks, task details, and queue list.

To display details of FreeRTOS Task List, enable the macro as below in the FreeRTOSConfig.h.

Note: If you use the FreeRTOS BSP generated from BSP Editor, you do not need to manually configure configUSE_TRACE_FACILITY in FreeRTOSConfig.h. The setting depends on system.h (part of FreeRTOSConfig.h #include), which can be configured using BSP Editor GUI or niosv-bsp command.

Table 9. FreeRTOS Debug Configuration

Debug Information	Macros	Configuration	Description
Task List	configUSE_TRACE_FACILITY	1	To display task number of the project

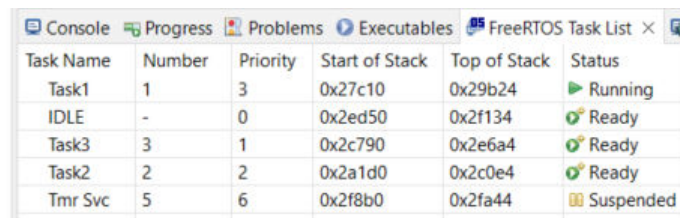
The configuration is as shown below:

```
#define configUSE_TRACE_FACILITY 1
```

To debug FreeRTOS application, follow these steps:

1. Launch the FreeRTOS application debug.
 - To open the FreeRTOS Task List, go to **Window > Show View > Other > FreeRTOS > FreeRTOS Task List**.

Figure 28. FreeRTOS Task List View

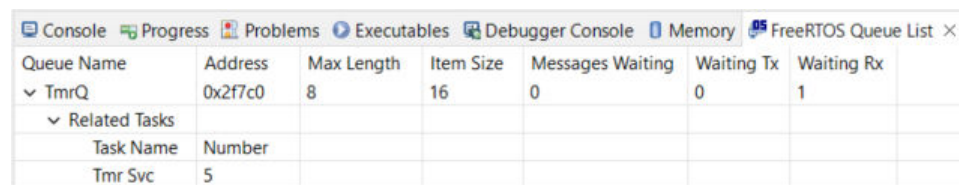


Task Name	Number	Priority	Start of Stack	Top of Stack	Status
Task1	1	3	0x27c10	0x29b24	Running
IDLE	-	0	0x2ed50	0x2f134	Ready
Task3	3	1	0x2c790	0x2e6a4	Ready
Task2	2	2	0x2a1d0	0x2c0e4	Ready
Tmr Svc	5	6	0x2f8b0	0x2fa44	Suspended

The **FreeRTOS Task List** shows task details including task name, priority, status, and stack details.

- To open the FreeRTOS Task List, go to **Window > Show View > Other > FreeRTOS > FreeRTOS Task List**

Figure 29. FreeRTOS Queue List View



Queue Name	Address	Max Length	Item Size	Messages Waiting	Waiting Tx	Waiting Rx
▼ TmrQ	0x2f7c0	8	16	0	0	1
▼ Related Tasks						
Task Name	Number					
Tmr Svc	5					

The **FreeRTOS Queue List View** shows details of all events and queues, including tasks related to each event.

Related Information

[FreeRTOS Real-Time Operating System section, Nios V Processor Software Developer Handbook](#)

For more information about FreeRTOS.

6.6. Debugging Zephyr Application

When you debug the Nios V processor application, you can view the Zephyr Mutex (Mutual Exclusion Object) List, Zephyr Semaphore List, and Zephyr Thread List.

To display details of Zephyr Mutex, Semaphore and Thread lists, enable the macros as below in the `prj.conf` file.

Table 10. Zephyr Debug Configuration

Debug Information	Macros	Configuration	Description
Enable Trace	CONFIG_TRACING	y	To enable all the tracing function. <i>Note:</i> All tracing macros have dependency to CONFIG_TRACING.
Mutex List	CONFIG_TRACING_OBJECT_TRACKING	y	To enable object tracking for mutex.
Semaphore List			To enable object tracking for semaphore.
Thread List	CONFIG_DEBUG_THREAD_INFO	y	To display thread details.

The configuration is as shown below:

```
CONFIG_TRACING=y
CONFIG_TRACING_OBJECT_TRACKING=y
CONFIG_DEBUG_THREAD_INFO=y
```

To debug Zephyr application, follow these steps:

1. Launch the Zephyr application debug.
 - To open the Zephyr Mutex List view, go to **Window > Show View > Other > Zephyr Mutex List**.

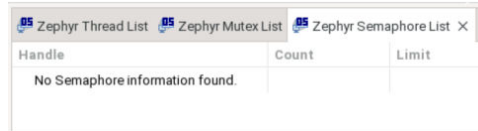
Figure 30. Zephyr OS Mutex List View

Handle	Thread Owner	Lock Count	Priority
0x88a0	Philosopher 4	1	-1
0x8888	Philosopher 4	1	-1
0x8870	Philosopher 3	1	0
0x8858	0x0	0	2
0x8840	Philosopher 0	1	-2
0x8828	Philosopher 0	1	3

The **Zephyr Mutex List** tab shows details of all mutexes including handle, thread owner, lock count, priority, along with any wait threads related to each mutex.

- To open the Zephyr Semaphore List view, go to **Window > Show View > Other > Zephyr Semaphore List**.

Figure 31. Zephyr Semaphore List View



The **Zephyr Semaphore List** tab shows details of all semaphores including handle, count, limit along with any wait threads related to each semaphore.

- To open the Zephyr Thread List view, go to **Window > Show View > Other > Zephyr Thread List**.

Figure 32. Zephyr OS Thread List View

Name	Handle	Priority	Start of Stack	Stack Size	Status
Philosopher 5	0x85d0	-2	0xb0f0	2048	Pending
Philosopher 4	0x8508	-1	0xa8f0	2048	Running
Philosopher 3	0x8440	0	0xa0f0	2048	Queued
Philosopher 2	0x8378	1	0x98f0	2048	Suspend
Philosopher 1	0x82b0	2	0x90f0	2048	Suspend
Philosopher 0	0x81e8	-2	0x88f0	2048	Suspend
idle	0x8698	40	0xc0f0	512	Dummy

The **Zephyr Thread List** tab shows thread details including name, handle, priority, start of stack, stack size, and status details.

Related Information

[Nios V Processor Zephyr Design Example RocketBoards](#)

6.7. Arm HPS On-Chip Trace

You can configure and capture the Arm HPS On-Chip Trace with Intel FPGA Download Cable II. The following table shows the Arm trace sources that generate instruction trace:

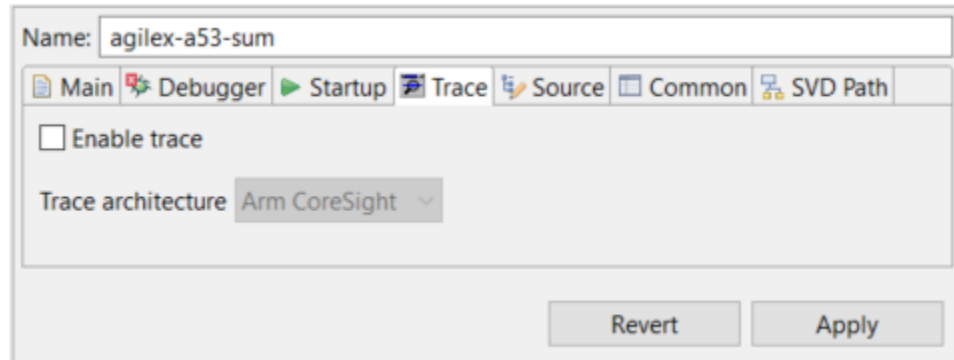
Table 11. Arm Trace Source

Arm Trace Source	Encoding Type	Description	Supported Device
Program Trace Macrocell (PTM)	PTM	A real-time trace module providing instruction tracing of a processor	- Intel Arria 10 - Intel Cyclone V
Embedded Trace Macrocell (ETM)	ETMv4	The Arm trace source that captures data and instruction trace	- Intel Agilex 7 - Intel Stratix 10

Follow the steps to enable and configure the **Trace** feature:

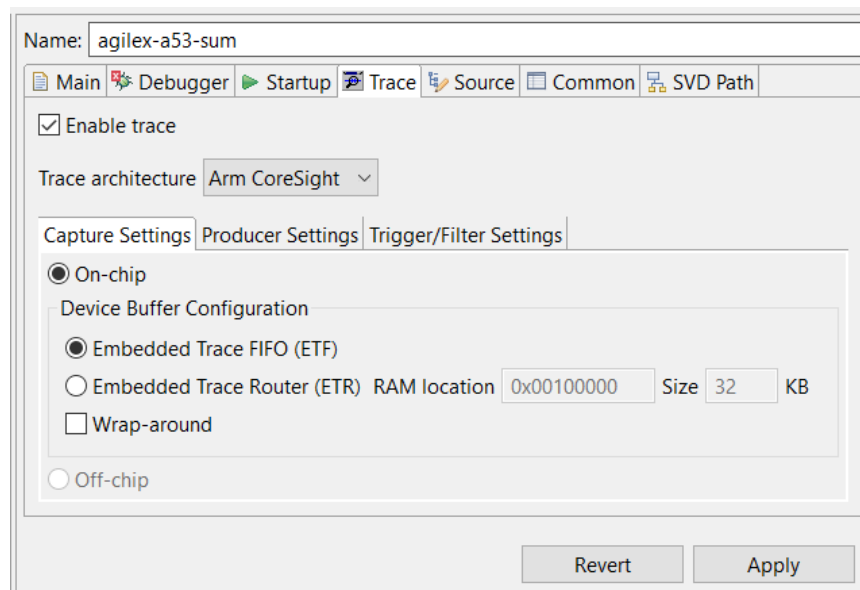
1. Launch **Ashling Arm Hardware Debugging**, and click the **Trace** tab.

Figure 33. Trace Tab



2. Click the **Enable trace** checkbox to enable the trace selected settings. The **Trace architecture** defaults to **Arm Coresight**. The trace selected settings appear in the following tab:
 - a. **Capture Settings**
 - b. **Producer Settings**
 - c. **Trigger/Filter Settings**

Figure 34. Capture Settings Sub-Tab



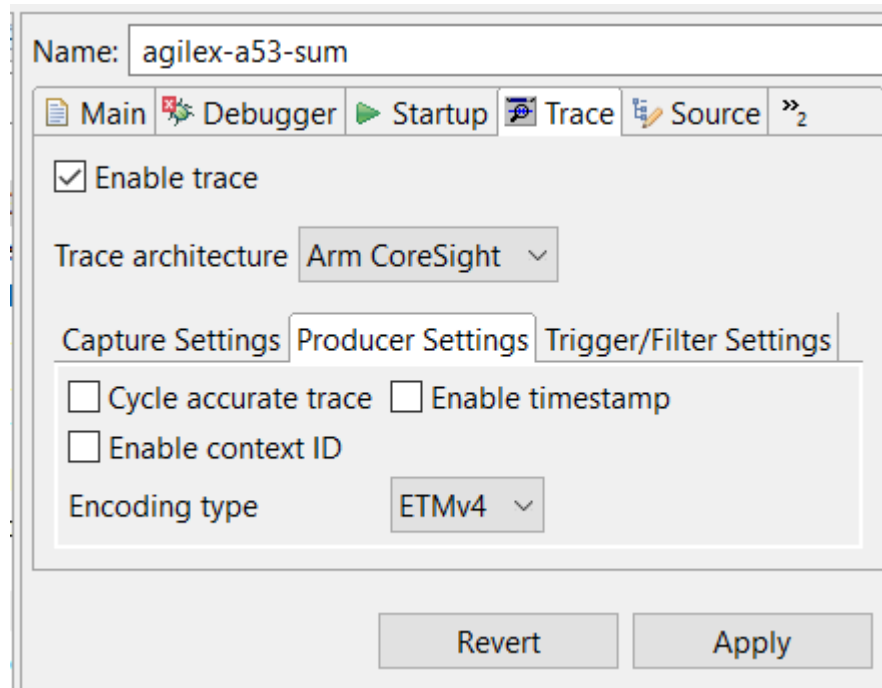
3. Under the **Capture Settings** tab, set this settings:
 - a. Select **On-chip**.
 - b. Select one of the following Device Buffer Configuration:

- i. **Embedded Trace FIFO (ETF)** — stores trace data in small and fixed size trace buffer.
 - ii. **Embedded Trace Router (ETR)** — routes the trace data to the system memory. You need to specify the RAM location and Size .
- c. **Wrap-around** — enables overwriting old trace data with the new trace data when the trace buffer is full.

Note: **Wrap-around** is optional. If you do not select **Wrap-around**, trace capture stops when the trace buffer is full.
4. Under **Producer Settings** tab, set this settings:
 - a. Select your preferred settings:
 - i. **Cycle Accurate Trace** — captures trace on all cycles even if there is no trace to output on that cycle.
 - ii. **Enable context ID** — enables the tracing of the ASID and the current process ID.
 - iii. **Enable timestamp** — enables timestamps in your trace.
 - b. For **Encoding type**, you can select **ETMv4** or **PTM**. Refer to Table [Arm Trace Source](#) for more details.

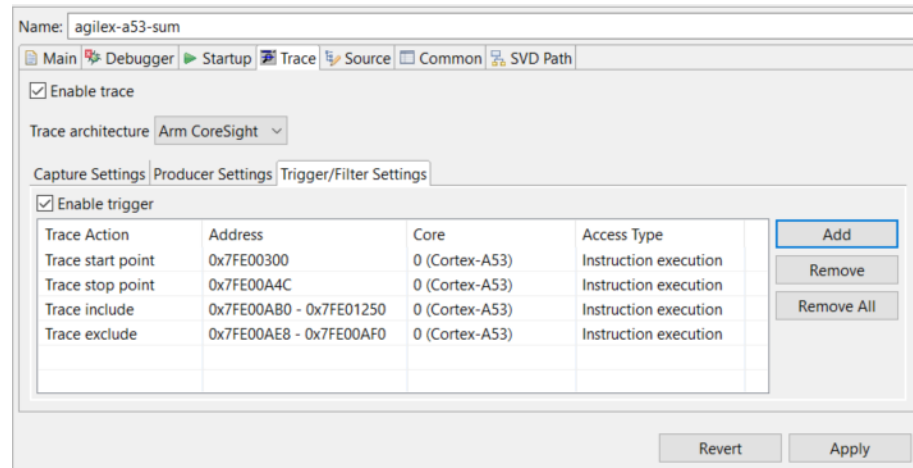
Note: **ETMv3** is an older version of **ETMv4** and not supported by Intel devices.

Figure 35. Producer Settings Sub-tab



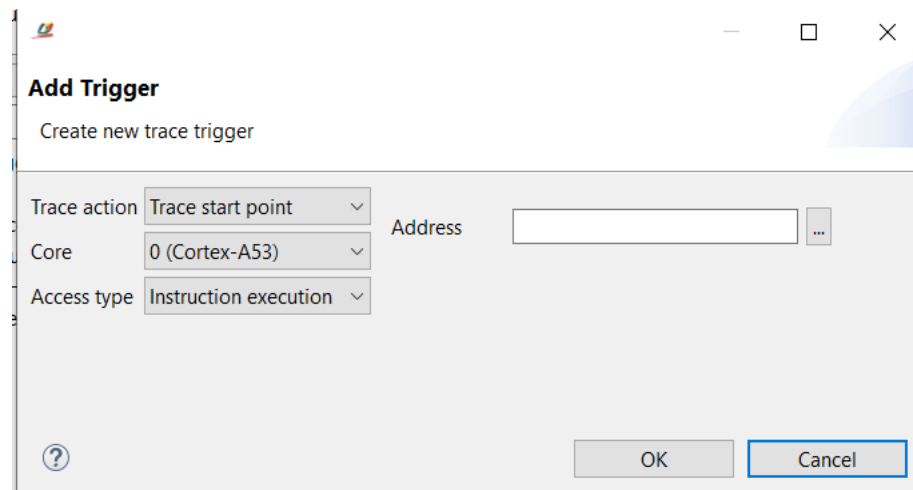
5. Under **Trigger/Filter Settings** tab, click the **Enable trigger** checkbox.

Figure 36. Trigger/Filter Settings Tab



6. Click **Add** to bring up the **Add Trigger** dialog box as shown below:

Figure 37. Add Trigger Dialog Box



- a. For **Trace action**, choose one of the following options:

- i. **Trace include**— limits trace to the selected address range
- ii. **Trace exclude**— excludes trace from the selected address range
- iii. **Trace start point**— starts tracing when the selected address is executed
- iv. **Trace stop point**— stops tracing when the selected address is executed
- b. For **Core**, use the same core that you select in Debugger tab.
- c. For **Access type**, select the only available option — **Instruction execution** as instruction trace and to support trigger.
7. Once you complete the configuration, click **Debug**.
8. Open **Trace view** via **Window > Show > View > Other > Debug > Trace**. Click **Fetch, decode and populate trace data** button on the top right of the window to fetch trace data from the target and display in the Trace view as shown below.

Figure 38. Trace View

Timestamp	Cycles	Context Id	Core No	Address	OpCode	Disassembly	Source Line
246623777	482	0	0	0x000000007fe004ac	e1 1f 40 b9	ldr w1, [sp, #28]	sum += i;
246623777	---	0	0	0x000000007fe004b0	e0 1b 40 b9	ldr w0, [sp, #24]	sum += i;
246623777	---	0	0	0x000000007fe004b4	20 00 00 0b	add w0, w1, w0	sum += i;
246623777	---	0	0	0x000000007fe004b8	e0 1f 00 b9	str w0, [sp, #28]	sum += i;
246623777	---	0	0	0x000000007fe004bc	e0 1b 40 b9	ldr w0, [sp, #24]	for(int i = start; i <= end; i++)
246623777	---	0	0	0x000000007fe004c0	00 04 00 11	add w0, w0, #0x1	for(int i = start; i <= end; i++)
246623777	---	0	0	0x000000007fe004c4	e0 1b 00 b9	str w0, [sp, #24]	for(int i = start; i <= end; i++)
246623777	---	0	0	0x000000007fe004c8	e1 1b 40 b9	ldr w1, [sp, #24]	for(int i = start; i <= end; i++)
246623777	---	0	0	0x000000007fe004cc	e0 0b 40 b9	ldr w0, [sp, #8]	for(int i = start; i <= end; i++)
246623777	---	0	0	0x000000007fe004d0	3f 00 00 0b	cmp w1, w0	for(int i = start; i <= end; i++)
246623777	---	0	0	0x000000007fe004d4	cd fe ff 54	b.le 0x7fe004ac <sum_cortex_a53+28>	for(int i = start; i <= end; i++)
246624313	536	0	0	0x000000007fe004ac	e1 1f 40 b9	ldr w1, [sp, #28]	sum += i;
246624313	---	0	0	0x000000007fe004b0	e0 1b 40 b9	ldr w0, [sp, #24]	sum += i;
246624313	---	0	0	0x000000007fe004b4	20 00 00 0b	add w0, w1, w0	sum += i;
246624313	---	0	0	0x000000007fe004b8	e0 1f 00 b9	str w0, [sp, #28]	sum += i;
246624313	---	0	0	0x000000007fe004bc	e0 1b 40 b9	ldr w0, [sp, #24]	for(int i = start; i <= end; i++)
246624313	---	0	0	0x000000007fe004c0	00 04 00 11	add w0, w0, #0x1	for(int i = start; i <= end; i++)
246624313	---	0	0	0x000000007fe004c4	e0 1b 00 b9	str w0, [sp, #24]	for(int i = start; i <= end; i++)
246624313	---	0	0	0x000000007fe004c8	e1 1b 40 b9	ldr w1, [sp, #24]	for(int i = start; i <= end; i++)
246624313	---	0	0	0x000000007fe004cc	e0 0b 40 b9	ldr w0, [sp, #8]	for(int i = start; i <= end; i++)
246624313	---	0	0	0x000000007fe004d0	3f 00 00 0b	cmp w1, w0	for(int i = start; i <= end; i++)
246624313	---	0	0	0x000000007fe004d4	cd fe ff 54	b.le 0x7fe004ac <sum_cortex_a53+28>	for(int i = start; i <= end; i++)
246624797	483	0	0	0x000000007fe004ac	e1 1f 40 b9	ldr w1, [sp, #28]	sum += i;
246624797	---	0	0	0x000000007fe004b0	e0 1b 40 b9	ldr w0, [sp, #24]	sum += i;

6.7.1. Trace Bar

You can control the trace process using the **Trace Bar**.

Figure 39. Trace View Bar Options



Table 12. Trace Control Using the Trace Bar

Trace Bar	Function	Description
1	Fetch Trace	Fetches the trace data from the target trace buffer, decode and populate it in the Trace view.
2	Clear Trace	Clears the trace view and the target trace buffer data.
3	First Page	Navigates the trace data when there are multiple pages of trace information.
4	Previous Page	
5	Next Page	
6	Last Page	
7	Start Capture	Starts the trace capturing. Applicable only when you manually stop the trace using Stop Capture.
8	Stop Capture	Stops the trace.

6.7.2. Trace View

Table 13. Trace View Information and Description

Trace Information	Description
Timestamp	The timestamp value received from the target
Cycles	The CPU cycle count information. The cycle count is available from the target for a set of instructions only. Cycle count against an instruction represents the number of CPU cycles consumed for the set of instructions starting from the last cycle count received.
Context ID	Context ID of the current execution. Context ID gives the information about the ASID and the current Process ID.
Core No	Core index of the core executing the trace data.
Address	The instruction address executed by the CPU.
Opcode	Opcode of the executed instruction.
Disassembly	Disassembly of the executed instruction.
Source Line	Source Line corresponding to the instruction execution.

Example Use Case: Disassembly View

Double click the trace row to highlight the corresponding source line and disassembly in **Source** and **Disassembly** view. Refer Figure [Trace View](#) and the following figures for more details.

Figure 40. Source Highlighting – Debug and Disassembly Tab

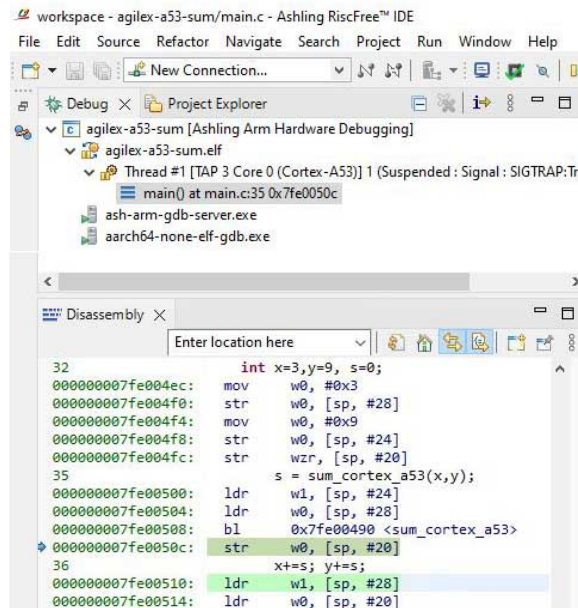
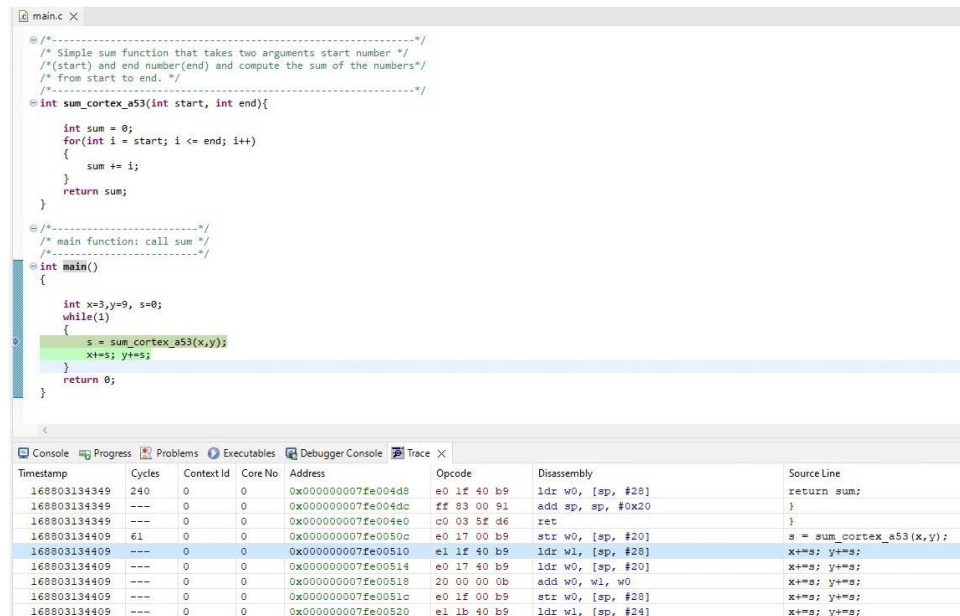


Figure 41. Source Highlighting – main.c Tab





7. Ashling RiscFree Integrated Development Environment (IDE) for Intel FPGAs User Guide Archives

For the latest and previous versions of this user guide, refer to [Ashling RiscFree Integrated Development Environment \(IDE\) for Intel FPGAs User Guide](#). If an IP or software version is not listed, the user guide for the previous IP or software version applies.

8. Document Revision History for the Ashling RiscFree Integrated Development Environment (IDE) for Intel FPGAs User Guide

Document Version	Intel Quartus Prime Version	Changes
2023.07.20	23.2	- Added a link to <i>AN 980: Nios® V Processor Intel® Quartus® Prime Software Support</i> in the topic <i>Intel® Quartus® Prime Software Support</i>. - Updated Table: <i>RiscFree IDE Open-Source Components</i> in <i>Open-Source Components in RiscFree IDE</i>.
2023.04.10	23.1	- Edited topic <i>Debug Setup for Nios V Processor System</i> to add the mention of Zephyr and FreeRTOS. - Updated the following figures: — <i>Debug Configurations for Nios V Processor —Debugger Tab</i> — <i>Debugger Settings for Intel Agilex 7 Arm Cortex-A53 Core</i> — <i>List of Available Cores for the Selected Target Device in RiscFree IDE</i> — <i>Startup Tab Settings</i> — <i>Enabling OS Aware Debugging in Heterogeneous Debugging</i> - Added new topics: — <i>Debugging FreeRTOS Application</i>. — <i>Debugging Zephyr Application</i>. - Added Ashling RiscFree IDE for Intel FPGAs and OpenCSD to the table <i>RiscFree IDE Open-Source Components</i>. - Updated product family name to Intel Agilex 7.
2022.12.19	22.4	- Added new topics in <i>Debugging with RiscFree IDE</i>. — <i>Arm HPS On-Chip Trace</i> — <i>Trace Toolbar</i> — <i>Trace View</i> - Updated Table: <i>RiscFree IDE Open-Source Components</i> in <i>Open-Source Components in RiscFree IDE</i>.
2022.10.31	22.1std	- Added more devices to the <i>Supported Devices</i> section. - Added links to the documentations supporting <i>Intel Quartus Prime Standard Edition</i> in <i>Supported Devices</i> section. - Updated references from <i>Intel Quartus Prime Pro Edition</i> to <i>Intel Quartus Prime</i> to indicate support for both Pro and Standard Edition. - Added steps to download Ashling RiscFree IDE for Intel FPGAs in the <i>Standalone Installation</i> section. - Added a figure <i>Installation Warning Message</i> in <i>Standalone Installation with Supported Intel Quartus Prime Software</i> section.
2022.09.26	22.3	- Updated the document title from <i>RiscFree Integrated Development Environment (IDE) for Intel FPGAs User Guide</i> to <i>Ashling RiscFree Integrated Development Environment (IDE) for Intel FPGAs User Guide</i>. - Updated the features in the <i>About the RiscFree IDE</i> section. - Added a note in the <i>Standalone Installation with the Intel Quartus Prime Pro Edition Programmer and Tools</i> session.

continued...

Document Version	Intel Quartus Prime Version	Changes
		- Updated the RiscFree IDE launching method and added a note to avoid the <code>niosv-stack-report</code> warning for standalone installation in the <i>Command Line Interface</i> section. - Added information on Nios V applications OS support in the <i>Debug Setup for Nios V Processor System</i> section. - Update the <i>Setting Debug Configurations and Downloading Nios V Project Using RiscFree IDE</i> section. - Updated the <i>Debug Configurations for Nios V—Main Tab</i> diagram. - Added step on OS Awareness settings. - Added note to state that RiscFree IDE supports Arm HPS standalone debug in the <i>Debug Setup for Arm Hard Processor System</i> section. - Updated the <i>Setting Core Configuration</i> section. - Updated the <i>Target Application Tab Settings</i> diagram. - Added step on OS Awareness settings. - Added the following sections: <i>Debugging μC/OS-II Application</i> <i>Help Content</i> <i>Open-Source Components in RiscFree IDE</i>
2022.06.21	22.2	Initial release.

A. Appendix

A.1. Optional Toolchain Configuration

A.1.1. Toolchain for Application and BSP

The `risc32-unknown-elf` toolchain is included in RiscFree IDE by default. Toolchain configuration is not required to build the software for current Nios V processor system.

To build an application or BSP with a toolchain other than `risc32-unknown-elf`, perform these configuration steps to include the other toolchain to the build path:

Note: If you configure the BSP setting file to use the RiscFree IDE toolchain (`risc32-unknown-elf`), do not perform this configuration.

Follow the steps below to include the toolchain in path of the RiscFree IDE builder:

1. Download the open source GNU RISC-V Embedded GCC package version v10.2.0-1.2 or later that supports the necessary RISC-V Instruction-Set Architecture (ISA).
2. Extract the package in `<Intel Quartus Prime installation directory>/niosv/` directory. Use the same directory for Linux and Windows.
3. Right click the project under the **Project Explorer** section and click **Properties**.
4. Go to **C/C++ Build > Environment**. Double click **PATH variable** to add the toolchain path. Example on adding `xpack-riscv-none-embed-gcc` toolchain:
 - Linux (add ":" as a separator for multiple path variables):
`<Intel Quartus Prime installation directory>/niosv/xpack-riscv-none-embed-gcc-10.2.0-1.2/bin`
 - Windows (add ";" as a separator for multiple path variables):
`<Intel Quartus Prime installation directory>/niosv/xpack-riscv-none-embed-gcc-10.2.0-1.2/bin`
5. Click **OK** and **Apply & Close**.

A.1.2. Toolchain for Arm Processor

Only the Nios V toolchain is included in RiscFree IDE. The CDT plugins from Eclipse support cross-building for Arm.

To use CDT plugins for Arm using your own toolchain, follow these steps:

1. Go to **File > New > C/C++ Project** and click **Next**.
2. Select **C Managed Build** and click **Next**.
3. Select project type. The project type can be completely managed by CDT, CMake, or make-based.
4. Enter Project Name as **Test Bare-Metal** or your preferred name. Select Project type as **Executable > Hello World C++ Project**, and Toolchain as **Cross GCC**. Click **Next**.
5. Enter Author, Copyright notice, and other details. Click **Next**. This step is optional.
6. Ensure **Debug** and **Release** options are enabled. Click **Next**.
7. Enter the cross GCC details as follows:

Devices	Intel Arria 10 and Cyclone V (Non-SDM Devices)	Intel Stratix 10 and Intel Agilex 7 (SDM Devices)
Cross compiler prefix	arm-none-linux-gnueabihf-	aarch64-none-linux-gnu-
Cross compiler path	<path_to_toolchain_directory>/gcc-arm-11.2-2022.02-x86_64-arm-none-linux-gnueabihf/bin/	<path_to_toolchain_directory>/gccarm-11.2-2022.02-x86_64-aarch64-none-linux-gnu/bin/

Note: The toolchain varies for SDM and non-SDM devices. For details on the toolchain, refer to GSRD of the device you use.

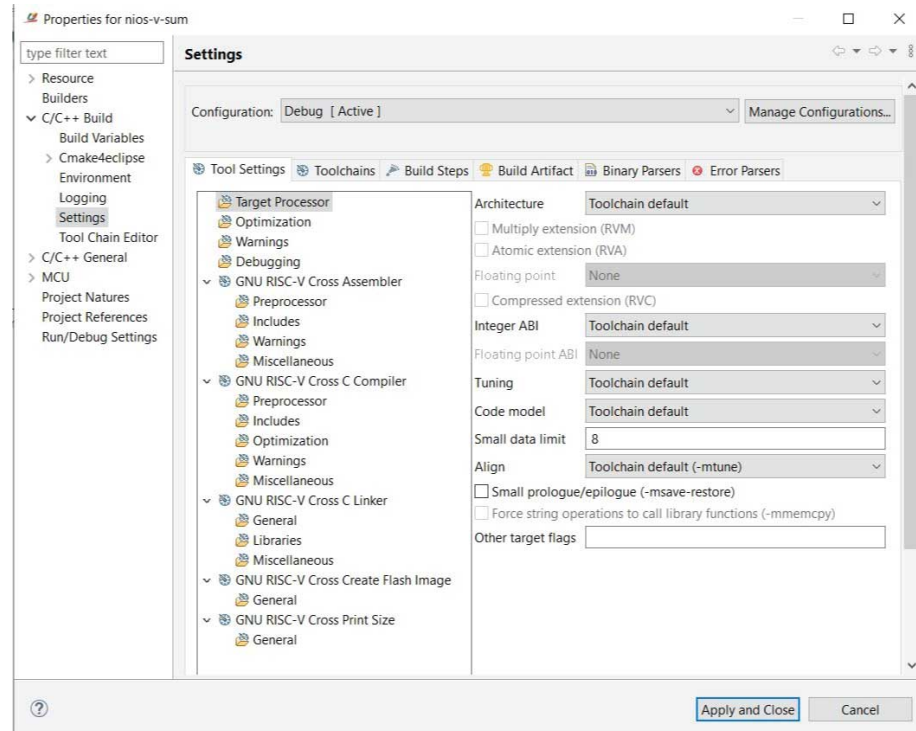
8. Click **Finish**. The project is created.

A.1.3. Modifying Toolchain Configuration

To change the toolchain options, follow these steps:

1. Right click project files (application or BSP) and select **Properties**.
2. Click **C/C++ Build > Settings**.

Figure 42. Modifying Toolchain Configuration



A.2. Performing External Tools Configuration

External tool configuration is a generic feature where you can configure the RiscFree IDE to include tools based on your requirement.

To perform external tool configuration for juart terminal, follow these steps:

1. Go to **Run ► External Tools ► External Tools Configurations....** Double click **Program** to open a **New_configuration** window.
2. Rename the configuration as **Nios V JTAG UART Output**.
3. Browse and select the `juart-terminal` file in the following paths:

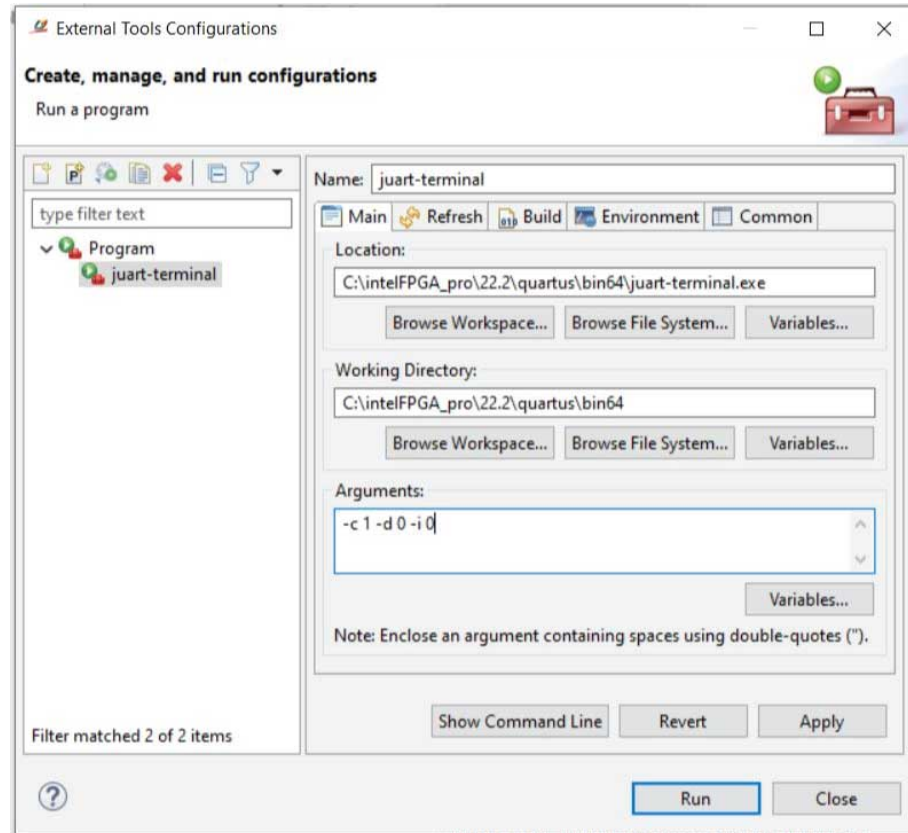
Operating System		Path
Windows	Intel Quartus Prime Software	<Intel Quartus Prime installation directory>/quartus/bin64/juart-terminal.exe
	Intel Quartus Prime Programmer and Tools	<Intel Quartus Prime installation directory>/qprogrammer/quartus/bin64/juart-terminal.exe
Linux	Intel Quartus Prime Software	<Intel Quartus Prime installation directory>/quartus/linux64/juart-terminal
	Intel Quartus Prime Programmer and Tools	<Intel Quartus Prime installation directory>/qprogrammer/quartus/linux64/juart-terminal

4. Set the preferred arguments. In this example, set the arguments as follows:

- Connect to JTAG UART at cable 1: **-c 1**
- Connect to device 0: **-d 0**
- Connect to instance 0: **-i 0**

Note: JTAG device chain index varies according to your setup. To obtain the latest device chain index numbering, run `openocd-cfg-gen --print` command in Nios V Command Shell.

Figure 43. External Tools Configuration



Note: The steps are also applicable to other Nios V tools (`niosv-app` and `niosv-bsp`) under `<Intel Quartus Prime installation directory>/niosv/bin` directory. These tools are not available for standalone RiscFree IDE installation with the Intel Quartus Prime Programmer and Tools.

A.3. Help Content

For more information about RiscFree IDE, go to **Help ► Help Content**. You can use the search bar to find the information about a specific topic.

A.4. Open-Source Components in RiscFree IDE

The following table shows the open-source components bundled with the RiscFree IDE.

Table 14. RiscFree IDE Open-Source Components

Open-Source Component	Version Number	Source Code Link	Tag
Eclipse CDT (C/C++ Development Tooling)	10.7.1	https://github.com/AshlingMicrosystems/cdt	cdt-RFI-v23.1.1
Eclipse Embedded CDT	6.3.1	https://github.com/AshlingMicrosystems/eclipse-plugins	eclipse-plugin-RFI-v23.2.1
cmake4eclipse	2.1.4	https://github.com/AshlingMicrosystems/cmake4eclipse	cmake4eclipse-RFI-v23.2.1
RISC-V GNU GCC	12.1.0	https://github.com/AshlingMicrosystems/gcc	gcc-RFI-v23.2.1
RISC-V GNU Binutils	2.39	https://github.com/AshlingMicrosystems/binutils-gdb	binutils-RFI-v23.2.1
RISC-V GNU GDB	12.1	https://github.com/AshlingMicrosystems/binutils-gdb	gdb-RFI-v23.2.1
RISC-V Newlib	4.1.0	https://github.com/AshlingMicrosystems/newlib	newlib-RFI-v23.2.1
RISC-V Picolibc	1.7.7	https://github.com/AshlingMicrosystems/picolibc	picolibc-RFI-v23.2.1
QEMU (Arm and RISC-V)	7.2.0	https://github.com/AshlingMicrosystems/qemu	qemu-RFI-23.2.1
OpenCSD	1.4.0	https://github.com/AshlingMicrosystems/OpenCSD	opencsd-RFI-23.2.1
Eclipse IDE	2022-09, 4.23	—	—
GNU Make	4.4	—	—
CMake	3.25.2	—	—
GNU Arm 32-bit GDB (arm-none-eabi 32-bit)	12.1	—	—
GNU Arm 64-bit GDB (aarch64-elf 64-bit)	12.1	—	—
Java Runtime Environment (OpenJDK)	v19	—	—